DS601PC: AUTOMATA THEORY AND COMPILER DESIGN

Lecture-12: Introduction Compiler Design

Dr. Anjeneya Swami Kare
Assistant Professor
School of Computer and Information Sciences
askcs@uohyd.ac.in

Syllabus

- UNIT - IV
  - Introduction: The structure of a compiler
  - Lexical Analysis: The Role of the Lexical Analyzer, Input Buffering, Recognition of Tokens, The Lexical- Analyzer Generator Lex
  - Syntax Analysis: Introduction, Context-Free Grammars, Writing a Grammar, Top-Down Parsing, Bottom- Up Parsing, Introduction to LR Parsing: Simple LR, More Powerful LR Parsers
- UNIT - V
  - Syntax-Directed Translation: Syntax-Directed Definitions, Evaluation Orders for SDD's, Syntax-Directed Translation Schemes, Implementing L-Attributed SDD's.
  - Intermediate-Code Generation: Variants of Syntax Trees, Three-Address Code
  - Run-Time Environments: Stack Allocation of Space, Access to Nonlocal Data on the Stack, Heap Management

Phases of Compiler

- Lexical Analysis (Scanner)
  - It reads the source program character by character, Converts the sequence of characters into tokens.
- Syntax Analysis (Parser)
  - Checks whether the token sequence follows grammar rules of the programming language.
  - Generates a Parse Tree or Syntax Tree.
- Semantic Analysis
  - Checks the meaning of the program. Ensures statements are semantically correct.
    - Type checking
    - Variable declaration verification
    - Function argument checking
    - Maintaining symbol table

Phases of Compiler

- Intermediate Code Generation
  - Produces an intermediate representation (IR) of the program.
- Code Optimization
  - Improves the intermediate code to make it faster and efficient.
  - Removes unnecessary operations.
  - Constant folding
  - Dead code elimination
  - Common subexpression elimination
  - Loop optimization
- Code Generation
  - Converts optimized intermediate code into target machine code

Lexical Analysis

What is Lexical Analysis?

- Lexical Analysis is the first phase of a compiler.
- It reads the source program as a stream of characters and groups them into meaningful sequences called tokens.
- These tokens are passed to the syntax analyzer (parser).
- The lexical analyzer converts characters → tokens.
- Example
  - int sum = a + b;

int → keyword
sum → identifier
= → operator
a → identifier
+ → operator
b → identifier
; → separator

Basic Terminology

- Token
  - A token is a pair consisting of token name and attribute value.
  - Example
    - <identifier, sum>
      <number, 25>
      <keyword, int>
- Pattern
  - A description of the form of tokens.
  - Example
    - Identifier pattern: letter (letter | digit)*
- Lexeme
  - The actual sequence of characters in the source program.
  - Example: sum, int, 25

Role of the Lexical Analyzer

- The lexical analyzer performs the following tasks:
  - Reads characters from source code
  - Groups characters into tokens
  - Removes whitespace and comments
  - Communicates with the symbol table
  - Detects lexical errors

Lexical Errors

- Lexical analyzer detects invalid tokens.
  - Invalid identifier
    - int 2value; Error: Identifier cannot start with digit.
  - Illegal character: a = b @ c; @ may be illegal in many languages.

Input Buffering

- Lexical analysis reads characters from the source program.
- Reading one character at a time is inefficient.
- Therefore Input Buffering is used.
- It improves speed of scanning.

Two-Buffer Scheme

- A common technique is Two Buffer Input Scheme.
  - Two buffers are used alternately.
  - Each buffer stores N characters.
  - Each buffer ends with a sentinel character (EOF).

Two Pointer Technique

- Two pointers are used:
  - LexemeBegin: Points to **start of lexeme**
  - Forward: Moves ahead to search for token end
- Example
  - Int count;

Recognition of Tokens

- Tokens are recognized using patterns.
- These patterns are described using Regular Expressions.
- Regular expressions are implemented using Finite Automata.

Symbol Table

- A Symbol Table is a data structure used by the compiler to store information about identifiers (such as variables, functions, classes, and constants) that appear in the source program.
- It is used during lexical analysis, syntax analysis, semantic analysis, and code generation to keep track of the properties of each identifier.
- A symbol table stores names of program entities and their attributes.

Symbol Table

- Each entry in the symbol table typically contains:

Field	Description
Name	Identifier name
Type	Data type of identifier
Scope	Where the identifier is valid
Memory Location	Address assigned
Value	Initial value (if constant)
Parameters	For functions

Symbol Table – Example-1

- int a;
  float b;
  int sum;

Identifier	Type	Scope	Memory Location
a	int	global	1000
b	float	global	1004
sum	int	global	1008

Symbol Table – Example-2

- int add(int x, int y)
  {
  int result;
  result = x + y;
  }

Identifier	Type	Scope	Additional Info
add	function	global	parameters: int, int
x	int	local	parameter
y	int	local	parameter
result	int	local	variable

Operations on Symbol Table

- The compiler performs several operations:
  - Insert: Add new identifier
  - Lookup: Find identifier information
  - Update: Modify attributes
  - Delete: Remove identifiers (when scope ends)

Why Symbol Table is Important?

- The symbol table helps the compiler to:
  - Detect undeclared variables
  - Type checking
  - Memory allocation
  - Scope management

Common Data Structures Used

- Symbol tables can be implemented using:
  - **Hash Table (most common)**
  - Binary Search Tree
  - Linked List
  - Balanced Tree

Machine Language Program Execution

- Instruction format
  - Size, Opcode and operand
- Role of registers
  - Program Counter (PC): Contains the address of next instruction to be executed
  - Accumulator (AC): To store intermediate results of arithmetic and logical operations
  - Instruction Register (IR): Contains the instruction that is currently being executed
  - Memory Address Register (MAR): Contains memory address of data
  - Memory Data Register (MDR): Contains the data that is being fetched from memory
  - General Purpose Registers: Additional registers that are present in the CPU which are used for either memory address or data
  - Condition Code Register (CCR): Contains the status or condition of the previous result from the ALU.

Machine Language Program Execution (Continued)

- Role of main memory
  - A sequence of bytes
  - Holds program under execution and data
- Role of cache memory
  - Faster memory that stores frequently accessed data and instructions
- Role of JUMP instruction
- Tracing of machine language programs
  - Addition of two numbers
  - Computing factorial of a number

Machine Language Programs

- Addition of Two Numbers
- 16 bit instruction, 4 bits Opcode and 12 bits operand

Sample Machine Language
0010 1010 0000 0100
0001 1010 0000 0101
0011 1010 0000 0110
0111 0000 0000 0001
1111 1111 1001 1100
0000 0000 0011 0010
0000 0000 0000 0000

Machine Language Programs (Continued)

- Addition of Two Numbers

Instruction Location (Decimal)	Instruction Location (Hexa)	Instruction (Binary)	Instruction (Hexa)	Assembly Language Instruction	Comment
2560	A00	0010 1010 0000 1000	2A08	LDA A08	Load Contents of A08 into AC
2562	A02	0001 1010 0000 1010	1A0A	ADD A0A	Add Contents of A0A to AC
2564	A04	0011 1010 0000 1100	3A0C	STA A0C	Store AC contents in A0C
2566	A06	0111 0000 0000 0001	7001	HLT	Halt Computer
2568	A08	1111 1111 1001 1100	FF9C	Operand	-100
2570	A0A	0000 0000 0011 0010	0032	Operand	50
2572	A0C	0000 0000 0000 0000	0000	Operand	Sum

Machine Language Programs (Continued)

- Factorial of a number

Instruction Location (Decimal)	Instruction Location (Hexa)	Instruction (Binary)	Instruction (Hexa)	Assembly Language Instruction	Comment
2560	A00	0010 1010 0001 0010	2A12	LDA A12	Load contents of A12
2562	A02	0100 1010 0001 0000	4A10	MLT A10	Multiply AC with contents of A10
2564	A04	0011 1010 0001 0010	3A12	STA A12	Store contents of AC at A12
2566	A06	0010 1010 0001 0000	2A10	LDA A10	Load Contents A10
2568	A08	0101 0000 0000 0000	5000	DEC	Decrement AC
2570	A0A	0011 1010 0001 0000	3A10	STA A10	Store contents of AC at A10
2572	A0C	0110 1010 0000 0000	6A00	JNZ A00	Jump to A00 if AC is not zero
2574	A0E	0111 0000 0000 0001	7001	HLT	Halt
2576	A10	0000 0000 0000 0100	0004	Operand	Value of n
2578	A12	0000 0000 0000 0001	0001	Operand	Location to store factorial of n

Tracing of Machine Language Programs

- Factorial of a number

Instruction Location (Decimal)	Instruction Location (Hexa)	Instruction (Binary)	Instruction (Hexa)	Assembly Language Instruction	Comment
2560	A00	0010 1010 0001 0010	2A12	LDA A12	Load contents of A12
2562	A02	0100 1010 0001 0000	4A10	MLT A10	Multiply AC with contents of A10
2564	A04	0011 1010 0001 0010	3A12	STA A12	Store contents of AC at A12
2566	A06	0010 1010 0001 0000	2A10	LDA A10	Load Contents A10
2568	A08	0101 0000 0000 0000	5000	DEC	Decrement AC
2570	A0A	0011 1010 0001 0000	3A10	STA A10	Store contents of AC at A10
2572	A0C	0110 1010 0000 0000	6A00	JNZ A00	Jump to A00 if AC is not zero
2574	A0E	0111 0000 0000 0001	7001	HLT	Halt
2576	A10	0000 0000 0000 0100	0004	Operand	Value of n
2578	A12	0000 0000 0000 0001	0001	Operand	Location to store factorial of n

Assembly Language Program

- Factorial of a number

Instruction No	Assembly Language Instruction
001	LDA F
002	MLT N
003	STA F
004	LDA N
005	DEC
006	STA N
007	JNZ 001
008	HLT
009	F, 1
010	N, 4

High Level Programming Languages

- C Program for factorial of a number

```
int main()
{
    int f = 1, n = 4;
    while(n>0)
    {
        f = f * n;
        n--;
    }
}
```

Lexical Analyzer Generator – Lex

- Lex is a tool used to automatically generate lexical analyzers (scanners).
- Instead of writing the lexical analyzer manually, we describe tokens using patterns, and Lex generates the scanner automatically.
- Lex was developed for Unix systems and is commonly used with YACC (Yet Another Compiler Compiler).

Purpose of Lex

- Automatically generating lexical analyzers
- Recognizing tokens using regular expressions
- Reducing manual coding effort
- Improving compiler development speed
- Easily modifying token rules

Simpletron High-Level Language (SHL)

- Instruction Set:
  - rem - any thing after rem is treated as a comment
    - Example: rem This is a sample program
  - input - to read value from the user
    - Example: input x
  - let – assignment statement
    - Examples:
      - let x = 2
      - let x = a + b
  - print - output statement
    - print x
    - print 23
  - goto – unconditional branch
    - Example: goto 50
  - if .. Goto
    - Example: if a == y goto 80

Simpletron High-Level Language (SHL)

- SHL variables:
  - SHL can have variables, variable name should be single letter alphabet
  - No declaration is needed, when the first time variable is seen its taken as declared with the value zero
  - SHL is case sensitive – all the instructions (keywords) are in lower case
  - SHL supports the following operators:
    - Arithmetic: +, -, *, /
    - Relational: <, >, <=, >=, ==, !=
  - Operator precedence is same as in C
  - Brackets () can be used to force the order of evaluation

Example SHL Program

Addition of Two Numbers: C = A + B

```
10  rem this is a program to compute C = A + B
11  input A
12  input B
13  let C = A + B
14  print C
15  end
```

Example SHL Program

Maximum of Two Numbers: C = Max(A, B)

```
1    rem maximum of two numbers
2    input s
3    input t
4    if s >= t goto 7
5    print t
6    goto 8
7    print s
8    end
```

Example SHL Program

Square of several integer

```
1    Rem calculate square of several integer
2    Input j
3    If j == -1 goto 7
4    Let k = j * j
5    Print k
6    Goto 2
7    end
```



Lex for SHL

- flex tokenizer.l
  - This creates lex.yy.c
- gcc lex.yy.c -o tokenizer
  - This creates the tokenizer executable
- ./tokenizer Prog.sml